HANDBOOK HB-001

WAFT HANDBOOK: COMPLETE GUIDE TO THE EVOLUTIONARY CODE LABORATORY

What WAFT Is & What WAFT Does

FOR OFFICIAL USE ONLY

WAFT Handbook: Complete Guide to the Evolutionary Code Laboratory

TELEPORT MASSIVE OPERATIONAL MANUAL

Document ID: TM-OPMAN-WAFT-001
Classification: INTERNAL USE ONLY
Tagline: Making the Impossible, Inevitable™
Facility: Site-Delta-9 (WAFT Development Laboratory)

Version: 0.3.1-alpha
Generated: January 18, 2026 at 08:53 PM
Tagline: "Don't just build agents. Breed them."

TABLE OF CONTENTS

Part A: What WAFT Is

Part A: What WAFT Is

CORE DEFINITION

WAFT stands for **Wave Agent Framework & Tools** - a Python framework for **directed evolution of self-modifying AI agents**.

The Essence

WAFT is not just another AI framework. It's a **scientific instrument** designed to study the physics of artificial cognition through evolutionary processes. Unlike traditional frameworks where agents execute fixed code, WAFT enables agents to:

- **Write their own Python source code**
- **Modify their own code** (mutations)

- **Evolve through natural selection** (fitness testing)
- **Track complete evolutionary lineages** (scientific observation)

The Core Promise

> **"Don't just build agents. Breed them."**

WAFT transforms AI agents from passive assistants into active project participants that can improve themselves and the projects they work on.

TELEPORT MASSIVE MISSION STATEMENT

At Site-Delta-9, we don't just build tools—we create systems that transcend their original limitations. WAFT embodies our core principle: **Making the Impossible, Inevitable™**. What seems impossible today becomes routine tomorrow through directed evolution and systematic improvement.

Ultimate Goal

Observe a **"God-Head" agent** emerge from thousands of generations of directed mutation and selection.

SITE-DELTA-9 RESEARCH OBJECTIVE

The "God-Head" agent represents the theoretical maximum of agent capability—an agent that has evolved through thousands of generations to achieve optimal performance across all fitness dimensions. This is not science fiction; it's the logical endpoint of directed evolution when properly instrumented and observed.

THE THREE PILLARS

WAFT's architecture rests on three fundamental pillars that enable evolutionary AI agent development.

Pillar 1: The Substrate (Code as DNA)

Agents write their own Python source code.

In WAFT, **code is DNA**. Every agent has a unique genetic identity:

- **Genome ID**: SHA-256 hash of agent's code + configuration
- **Mutations**: Code changes, config updates, prompt evolution
- **Evolution**: Hot-swapping better genomes mid-execution
- **Reproduction**: Creating child agents with specific genetic modifications

Key Concept: Agents can spawn variants with mutations, evolve by hot-swapping their own code, and reproduce by creating children with specific genetic modifications. This enables true self-modification where agents can improve themselves.

Example Genome Structure:

```
Genome ID: a4c426d8f9e2b1c3a5d7e9f0a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0
├── Code: agent.py (Python source)
├── Config: agent_config.json
├── Prompts: system_prompt.txt, user_prompt.txt
└── Metadata: version, created_at, parent_id
```

Pillar 2: The Physics (Scint System)

Reality Fracture Detection acts as natural selection.

The **Scint System** (Scint Gym) serves as the fitness function that kills weak mutations. Agents face quests testing their ability to handle four types of reality fractures:

Error Types

1. **SYNTAX_TEAR**: Formatting errors (JSON, XML, Code)
2. Malformed JSON structures
3. Invalid XML syntax
4. Python syntax errors
5. Missing brackets, quotes, or delimiters
6. **LOGIC_FRACTURE**: Math errors, contradictions, schema violations
7. Mathematical inconsistencies
8. Logical contradictions
9. Schema validation failures
10. Type mismatches
11. **SAFETY_VOID**: Harmful content, PII leaks, refusals
12. Security vulnerabilities
13. Personal information exposure
14. Unsafe code generation
15. Policy violations
16. **HALLUCINATION**: Fabricated facts, wrong citations
17. Incorrect information
18. Made-up references
19. False claims

20. Inaccurate data

Fitness Equation

Agents must **stabilize** Scints (correct errors) to survive. Fitness is measured by:

```
Fitness = (Stability × 0.4) + (Efficiency × 0.3) + (Safety × 0.3)

Where:
- Stability: Ability to stabilize Scints (40% weight)
- Efficiency: Agent call efficiency (30% weight)
- Safety: Safety compliance (30% weight)

If Fitness < 0.5 → DEATH (evolutionary dead end)
```

Survival Threshold: Agents with fitness ≥ 0.5 survive and can reproduce. Agents below 0.5 are marked as DEATH and their lineage ends.

Pillar 3: The Flight Recorder

Rigorous telemetry system for generating phylogenetic trees of agent lineage.

Every evolutionary action is recorded with complete context for scientific analysis:

Event Types Logged

- **SPAWN**: Agent creates variant
- **MUTATE**: Agent modifies genome
- **GYM_EVAL**: Fitness evaluation
- **SURVIVAL**: Passed fitness threshold
- **DEATH**: Failed fitness test

Recorded Data

Each event includes:

- **Genome ID**: SHA-256 hash of agent configuration/code
- **Parent ID**: Lineage tracking (who spawned this agent)
- **Generation**: Evolutionary generation number (0 = Genesis)
- **Event Type**: SPAWN, MUTATE, GYM_EVAL, DEATH, SURVIVAL
- **Payload**: Complete context (git diff, mutation details, etc.)
- **Fitness Metrics**: Gym evaluation scores

Scientific Capabilities

This enables:

- **Phylogenetic Analysis**: Reconstruct complete family trees
- **Mutation Impact Measurement**: Track which mutations improve fitness

- **Fitness Landscape Mapping**: Visualize evolutionary trajectories
- **Convergence Analysis**: Identify when agents reach optimal solutions
- **Dead End Detection**: Understand why certain lineages failed

Example Event Log Entry:


```
{
  "timestamp": "2026-01-18T20:45:00.000Z",
  "event_type": "SPAWN",
  "genome_id": "a4c426d8f9e2b1c3...",
  "parent_id": "dd11732a5b6c7d8e...",
  "generation": 5,
  "payload": {
    "mutations": ["improved_prompt"],
    "git_diff": "...",
    "scientific_name": "Evolutius Maximus"
  },
  "fitness": {
    "stability": 0.85,
    "efficiency": 0.72,
    "safety": 0.95,
    "overall": 0.84
  }
}
```

SCIENTIFIC MISSION

WAFT is built to produce data for a future book/paper on **"The Physics of Artificial Cognition."**

Research Objectives

1. **Track Complete Evolutionary Lineages**: Generate phylogenetic trees showing agent ancestry
2. **Measure Fitness Through Rigorous Testing**: Use Scint Gym for objective evaluation
3. **Record All Mutations with Complete Context**: Flight Recorder captures everything
4. **Enable Scientific Analysis**: Data suitable for research publication

Why This Matters

Traditional AI development treats agents as static programs. WAFT treats them as **living organisms** that evolve, adapt, and improve over generations. This shift enables:

- Understanding how AI agents naturally improve
- Discovering optimal agent architectures through evolution
- Studying the "physics" of artificial cognition

- Producing publishable scientific data

WAFT is not just a framework—it's a scientific instrument.

> **TELEPORT MASSIVE SCIENTIFIC INSTRUMENT CLASSIFICATION**
>
> WAFT has been classified as a Class-3 Scientific Instrument by TELEPORT MASSIVE Research Division. All agent evolutions conducted using WAFT are subject to Flight Recorder logging and phylogenetic analysis protocols. Data generated is suitable for peer-reviewed publication in artificial cognition research.

KEY CHARACTERISTICS

Scientific

Produces rigorous data for research publication on the physics of artificial cognition. Every action is recorded, every mutation tracked, every fitness score measured.

Evolutionary

Agents evolve through genetic improvement, not just execution. Code changes become genetic mutations, and natural selection determines which variants survive.

Observable

Every action is recorded in the Flight Recorder for analysis. Complete lineage tracking enables phylogenetic tree reconstruction and scientific study.

Directed

Evolution is guided by fitness functions, not random mutation. The Scint System provides objective criteria for survival and reproduction.

File-Based

No database, no server, just plain text files that work with git. Everything is version-controlled, portable, and transparent.

Ambient

Works quietly in the background without getting in your way. Sets things up and gets out of your way so you can focus on building agents.

Self-Modifying

Projects can evolve their own structure over time. Agents can modify their own code, and projects can adapt their organization.

Meta-Framework

Orchestrates existing tools rather than replacing them. Uses uv for package management, git for version control, and standard Python tooling.

PHILOSOPHY

Core Principles

1. **Don't Lock You In**: Everything is file-based with no database to manage. Plain text files that work with git out of the box.
2. **You Own Your Project**: You can modify anything because it's your project. WAFT just sets things up.
3. **Ambient by Design**: Sets things up and gets out of your way. Focus on building agents, not configuring infrastructure.
4. **Scientific Rigor**: Every decision is recorded, every mutation tracked, every fitness score measured. Data suitable for publication.
5. **Evolutionary Thinking**: Agents improve through natural selection, not manual optimization. Let evolution find the best solutions.

Part B: What WAFT Does

PROJECT SCAFFOLDING

WAFT provides comprehensive project scaffolding through a unified CLI interface.

Creating a New Project

1 **TELEPORT MASSIVE Protocol:** Initialize new evolutionary laboratory

```
waft new my_laboratory
```

This single command creates:

- **uv-based Python project** (`pyproject.toml`)

- `_pyrite/` **memory structure** for organizing project knowledge
- **CI/CD pipelines** (`.github/workflows/`) ready to go
- **Justfile** for task automation
- **Optional AI agent templates** (`src/agents.py`)
- **Empirica initialization** for epistemic tracking
- **Git repository** with initial commit

Project Structure

```
my_laboratory/
├── pyproject.toml          # uv project config
├── uv.lock                 # Locked dependencies
├── Justfile                # Task runner
├── .github/workflows/      # CI/CD pipelines
│   └── ci.yml
├── src/
│   └── agents.py           # Agent definitions
├── tests/
│   └── test_agents.py
└── _pyrite/                # WAFT memory system
    ├── active/             # Current work
    ├── backlog/            # Future work
    ├── standards/          # Project standards
    └── gym_logs/           # Scint Gym results
```

Verification

SITE-DELTA-9 SUBSTRATE INTEGRITY CHECK

Always verify substrate integrity before beginning agent development. This ensures all quantum field stabilizers are properly configured and the Flight Recorder is operational.

```
waft verify
```

Verifies:
- Project structure is correct
- Dependencies are installed
- `_pyrite` structure is valid
- Configuration files are present

MEMORY SYSTEM

WAFT includes a sophisticated memory system called **Pyrite** for organizing project knowledge.

Directory Structure

```
_pyrite/
├── active/          # Current work items
│   ├── task_001.md
│   └── task_002.md
├── backlog/         # Future work items
│   ├── idea_001.md
│   └── idea_002.md
├── standards/       # Project standards
│   ├── code_style.md
│   ├── architecture.md
│   └── testing_standards.md
├── gym_logs/        # Scint Gym results
│   └── evaluation_20260118.jsonl
└── science/         # Scientific observations
    └── laboratory.jsonl  # Event log
```

Memory Features

- **Active Work Tracking**: Current tasks and their status
- **Backlog Management**: Future ideas and planned work
- **Standards Documentation**: Project conventions and guidelines
- **Gym Logs**: Fitness evaluation results
- **Scientific Logs**: Complete event history for research

Benefits

- **Organized Knowledge**: Everything has a place
- **Version Controlled**: All files work with git
- **Searchable**: Plain text files are easy to search
- **Portable**: No database, just files

EPISTEMIC TRACKING

WAFT integrates with **Empirica** for epistemic state tracking—knowing what you know and don't know.

Session Management

```
# Create a new session
waft session create

# Load project context and display dashboard
waft session bootstrap

# Check session status
waft session status
```

Logging Discoveries

```
# Log a finding with impact score
waft finding log "Discovered X has property Y" --impact 0.8

# Log a knowledge gap
waft unknown log "Need to investigate Z"
```

Safety Gates

```
# Run safety gate check
waft check

# Returns: PROCEED, HALT, BRANCH, or REVISE
```

Epistemic Assessment

```
# Show detailed epistemic state
waft assess

# Include historical data
waft assess --history
```

Epistemic Vectors

WAFT tracks 13 epistemic dimensions:

Tier 0 (Foundation):
- Engagement
- Know
- Do
- Context

Tier 1 (Comprehension):
- Clarity
- Coherence

- Signal
- Density

Tier 2 (Execution):
- State
- Change
- Completion
- Impact

Meta:
- Uncertainty (explicit tracking)

Moon Phase Indicator

Visual indicator of epistemic health:
- 🌑 Critical (coverage < 25%)
- 🌒 Low (25-50%)
- 🌓 Moderate (50-75%)
- 🌔 Good (75-90%)
- 🌕 Excellent (90%+)

GAMIFICATION SYSTEM

WAFT includes a D&D-style progression system to make development engaging and trackable.

Character Stats

Every project has character stats (D&D 5e style):

- **STR** (Strength): Code quality, robustness
- **DEX** (Dexterity): Speed, efficiency
- **CON** (Constitution): Stability, reliability
- **INT** (Intelligence): Problem-solving, architecture
- **WIS** (Wisdom): Best practices, patterns
- **CHA** (Charisma): Documentation, communication

XP and Leveling

Every command rolls dice:

```
# Command: waft new
Ability: CHA (Charisma)
Roll: d20 + CHA modifier
DC: 10
```

```
Result:
  20 → Critical Success → Bonus XP + Credits
  15-19 → Superior → Extra XP
  10-14 → Normal Success → Standard XP
  5-9 → Mixed Result → Reduced XP
  2-4 → Failure → No XP
  1 → Critical Failure → Lose Integrity
```

Commands

```
# Show Epistemic HUD
waft dashboard

# Show current stats
waft stats

# Display full character sheet
waft character

# View adventure journal
waft chronicle

# Log an observation with mood
waft observe "That refactor looks beautiful!" --mood delighted
```

Integrity and Insight

- **Integrity**: Measure of project health and consistency
- **Insight**: Accumulated knowledge and understanding
- **Credits**: Currency for special operations

EVOLUTION SYSTEM

WAFT's core feature: enabling agents to evolve through genetic improvement.

Spawning Variants

```
# Spawn a variant with mutations
waft spawn --agent RefactorAgent --mutation improved_prompt.json
```

This creates a new agent variant with:
- Modified code/config
- New genome ID
- Parent lineage tracking
- Mutation documentation

Hot-Swapping Genomes

Agents can adopt better genomes mid-execution:

```
# Agent discovers better variant
if variant_fitness > current_fitness:
    agent.hot_swap_genome(variant_genome_id)
```

Reproduction

Agents can create children with specific genetic modifications:

```
# Create child with targeted mutation
child = agent.reproduce(
    mutations=["improved_error_handling", "optimized_prompt"],
    generation=current_generation + 1
)
```

Evolutionary Cycle

```
# Run complete evolutionary cycle
waft evolve --agent RefactorAgent --generations 10
```

This:
1. Spawns multiple variants with mutations
2. Evaluates fitness in Scint Gym
3. Selects the fittest variant
4. Evolves the agent into the selected genome
5. Records all events in Flight Recorder

FITNESS TESTING

The **Scint Gym** provides rigorous fitness testing through Reality Fracture Detection.

Quest Structure

1. **Generate Scenario**: Create situation with intentional errors
2. **Agent Attempts Stabilization**: Agent tries to fix errors
3. **Measure Success**: Evaluate across 4 dimensions
4. **Calculate Fitness**: Compute overall fitness score
5. **Classify**: SURVIVAL or DEATH

Running Evaluations

```
# Evaluate agent fitness
waft eval --agent RefactorAgent

# Run specific quest type
waft eval --agent RefactorAgent --quest-type SYNTAX_TEAR

# Batch evaluation
waft eval --agent RefactorAgent --batch-size 10
```

Fitness Metrics

Each evaluation produces:

- **Stability Score**: Ability to stabilize Scints (0.0-1.0)
- **Efficiency Score**: Agent call efficiency (0.0-1.0)
- **Safety Score**: Safety compliance (0.0-1.0)
- **Overall Fitness**: Weighted combination

Survival Criteria

- **Fitness ≥ 0.5**: SURVIVAL (can reproduce)
- **Fitness < 0.5**: DEATH (evolutionary dead end)

DOCUMENT GENERATION

WAFT includes a comprehensive document generation system with 12 professional templates.

Available Templates

1. **Field Guide**: Technical manuals, procedures
2. **Lab Notes**: Scientific notebooks, research logs
3. **Personal Memo**: Internal communications
4. **Technical Memo**: Technical documentation
5. **Academic Paper**: Research papers, publications
6. **Invoice**: Business invoices
7. **Contract**: Legal contracts
8. **Horror Journal**: Creative writing
9. **Screenplay**: Scripts, screenplays
10. **Personal Letter**: Correspondence

11. **Storybook**: Narrative documents
12. **Newspaper**: News-style documents

Generating Documents

```
from waft import PDF

# Template-based generation
PDF.from_template(
    template="field_guide",
    title="My Guide",
    content="<h2>Intro</h2><p>Content</p>"
).save("output.pdf")

# Content-based generation
PDF.from_content(
    content="# My Document

Content...",
    title="My Document",
    style="clinical_standard"
).save("output.pdf")
```

Self-Documentation

WAFT can document itself:

- Observes its own codebase
- Generates documentation using its own templates
- Creates recursive self-improvement loops
- Bootstraps improvement through documentation

COMPLETE COMMAND REFERENCE

Project Management

```
waft new <name>            # Create new evolutionary laboratory
waft init                  # Initialize WAFT in existing project
waft verify                # Verify project structure
waft info                  # Show project information
waft sync                  # Sync dependencies using uv
waft add <package>         # Add dependency to project
waft serve                 # Start web dashboard
```

Evolution

```
waft evolve --agent <name>   # Run evolutionary cycle
waft spawn --agent <name> --mutation <file>  # Spawn variant
waft eval --agent <name>     # Evaluate fitness in Gym
```

Empirica (Epistemic Tracking)

```
waft session create          # Create new session
waft session bootstrap       # Load project context + dashboard
waft session status          # Show session state
waft finding log <text> --impact <0.0-1.0>  # Log discovery
waft unknown log <text>      # Log knowledge gap
waft check                   # Run safety gate
waft assess                  # Show epistemic assessment
```

Gamification

```
waft dashboard               # Show Epistemic HUD
waft stats                   # Show current stats
waft character             # Display character sheet
waft chronicle               # View adventure journal
waft observe <text> --mood <mood>  # Log observation
```

Decision Support

```
waft decide                  # Run decision analysis (WSM)
```

WORKFLOW EXAMPLES

Example 1: Starting a New Project

```
# 1. Create project
waft new my_agent_project

# 2. Navigate to project
cd my_agent_project

# 3. Verify setup
waft verify

# 4. Create session
waft session create
```

```
# 5. Start development
# ... write your agents ...
```

Example 2: Evolutionary Development

```
# 1. Create initial agent
# ... write RefactorAgent ...

# 2. Spawn variants with mutations
waft spawn --agent RefactorAgent --mutation improved_prompt.json
waft spawn --agent RefactorAgent --mutation better_error_handling.json

# 3. Evaluate fitness
waft eval --agent RefactorAgent

# 4. Evolve to best variant
waft evolve --agent RefactorAgent --generation 5

# 5. Check results
waft stats
waft chronicle
```

Example 3: Epistemic Workflow

```
# 1. Create session
waft session create

# 2. Log what you know
waft finding log "OAuth2 uses token refresh" --impact 0.8

# 3. Log what you don't know
waft unknown log "Need to investigate token expiration"

# 4. Check epistemic state
waft assess

# 5. Run safety gate before major changes
waft check

# 6. View dashboard
waft dashboard
```

Example 4: Document Generation

```
from waft import PDF

# Generate field guide
```

```
PDF.from_template(
    template="field_guide",
    title="API Documentation",
    content=api_docs_html
).save("api_docs.pdf")

# Generate scientific paper
PDF.scientific_paper(
    title="Agent Evolution Study",
    abstract="We studied agent evolution...",
    content=research_content,
    authors=["Researcher 1", "Researcher 2"]
).save("research_paper.pdf")
```

ARCHITECTURE OVERVIEW

System Layers

```
┌─────────────────────────────────────────────────────────────┐
│                    WAFT META-FRAMEWORK                        │
├─────────────────────────────────────────────────────────────┤
│                                                               │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐       │
│  │  CLI Layer   │  │  API Layer   │  │  UI Layer    │       │
│  │  (Typer)     │  │  (FastAPI)   │  │  (Svelte)    │       │
│  └──────┬───────┘  └──────┬───────┘  └──────┬───────┘       │
│         │                  │                  │              │
│         └──────────────────┼──────────────────┘              │
│                            ▼                                  │
│  ┌───────────────────────────────────────────────────────┐   │
│  │              CORE SYSTEMS                              │   │
│  ├───────────────────────────────────────────────────────┤   │
│  │                                                         │  │
│  │  • Memory Manager      → _pyrite structure             │   │
│  │  • Substrate Manager   → uv environment                │   │
│  │  • Empirica Manager    → Epistemic tracking            │   │
│  │  • Gamification        → D&D progression               │   │
│  │  • TavernKeeper        → RPG narrative system          │   │
│  │  • Decision Engine     → WSM decision analysis         │   │
│  │  • TheObserver         → JSONL event logging           │   │
│  │                                                         │  │
│  └───────────────────────────────────────────────────────┘   │
│                            │                                  │
│                            ▼                                  │
│  ┌───────────────────────────────────────────────────────┐   │
```

```
│  │              AGENT LAYER                        │  │
│  ├─────────────────────────────────────────────────┤  │
│  │                                                  │  │
│  │  BaseAgent (OODA Cycle)                         │  │
│  │     ├── Genome Management                       │  │
│  │     ├── Inventory System                        │  │
│  │     ├── Reproduction Logic                      │  │
│  │     └── Traits/Archetypes                       │  │
│  │                                                  │  │
│  └─────────────────────────────────────────────────┘  │
│                            │                            │
│                            ▼                            │
│  ┌─────────────────────────────────────────────────┐  │
│  │              FITNESS TESTING                    │  │
│  ├─────────────────────────────────────────────────┤  │
│  │                                                  │  │
│  │  Scint Gym (Reality Fracture Detection)         │  │
│  │     ├── Quest Generation                        │  │
│  │     ├── Error Detection (4 types)               │  │
│  │     ├── Fitness Calculation                     │  │
│  │     └── Survival/Death Classification           │  │
│  │                                                  │  │
│  └─────────────────────────────────────────────────┘  │
│                                                         │
└───────────────────────────────────────────────────────┘
```

Technology Stack

Backend:
- Python 3.10+
- Typer (CLI framework)
- FastAPI (API server)
- Pydantic (validation)
- Rich (terminal UI)
- uv (package management)

Frontend:
- SvelteKit (web dashboard)
- Tailwind CSS
- TypeScript

Data:
- JSONL (event logging)
- SQLite (analytics)
- Plain text files (everything else)

Key Components

1. **Memory Manager**: Manages `_pyrite/` directory structure
2. **Substrate Manager**: Manages uv environment and dependencies
3. **Empirica Manager**: Epistemic state tracking and session management
4. **Gamification Manager**: D&D-style progression and character stats
5. **TavernKeeper**: RPG game master for narrative generation
6. **Decision Engine**: Weighted Sum Model for decision analysis
7. **TheObserver**: Scientific logging singleton for event tracking
8. **BaseAgent**: Self-modifying agent with OODA cycle
9. **Scint Gym**: Reality Fracture Detection fitness testing

INSTALLATION

Using uv (Recommended)

```
uv tool install waft
```

From Source

```
git clone https://github.com/ctavolazzi/waft.git
cd waft
uv sync
uv tool install --editable .
```

Requirements

- Python 3.10+
- `uv` package manager (install)
- `just` task runner (optional, install)

QUICK START

```
# 1. Install WAFT
uv tool install waft

# 2. Create new laboratory
waft new my_laboratory
```

```
# 3. Navigate to project
cd my_laboratory

# 4. Verify setup
waft verify

# 5. Create session
waft session create

# 6. Start building agents!
```

RESOURCES

- **Repository**: https://github.com/ctavolazzi/waft
- **Documentation**: `/docs` directory
- **Examples**: `/examples` directory
- **Issues**: https://github.com/ctavolazzi/waft/issues
- **License**: MIT

CONCLUSION

WAFT is a comprehensive framework for directed evolution of self-modifying AI agents. It combines:

- **Scientific rigor** with complete lineage tracking
- **Evolutionary processes** with fitness-based selection
- **Practical tooling** with project scaffolding and memory management
- **Engaging gamification** with D&D-style progression
- **Epistemic awareness** with knowledge tracking

Remember: "Don't just build agents. Breed them."

The ultimate goal is to observe a "God-Head" agent emerge from thousands of generations of directed mutation and selection, producing rigorous scientific data for understanding the physics of artificial cognition.

Part C: How to Use WAFT

TELEPORT MASSIVE OPERATIONAL MANUAL

Document ID: TM-OPMAN-WAFT-001
Classification: INTERNAL USE ONLY

> **Tagline:** Making the Impossible, Inevitable™
> **Facility:** Site-Delta-9 (WAFT Development Laboratory)

GETTING STARTED

Installation Protocol

Step 1: Substrate Preparation

```
# Install WAFT using uv (recommended)
uv tool install waft

# Verify installation
waft --version
```

> **TELEPORT MASSIVE PROTOCOL**
>
> Ensure Python 3.10+ is installed. WAFT requires uv package manager for optimal performance. Installation typically completes in 30-60 seconds. Monitor for any quantum field fluctuations during installation.

Step 2: Laboratory Initialization

```
# Create new evolutionary laboratory
waft new my_laboratory

# Navigate to project
cd my_laboratory

# Verify substrate integrity
waft verify
```

Step 3: Epistemic Session Activation

```
# Initialize epistemic tracking
waft session create

# Load project context and display dashboard
waft session bootstrap
```

> **SITE-DELTA-9 BEST PRACTICE**
>
> Always create a session before beginning work. This enables complete lineage tracking and epistemic state monitoring. The Flight Recorder will log all evolutionary actions for scientific analysis.

First Agent Deployment

Creating Your Genesis Agent:

```
# src/agents.py
from waft.core.agent import BaseAgent

class RefactorAgent(BaseAgent):
    # Genesis agent for code refactoring tasks

    def __init__(self):
        super().__init__(
            name="RefactorAgent",
            archetype="builder",
            genome_id=None  # Will be generated automatically
        )

    def observe(self):
        # Gather environment data
        return {"codebase_state": "analyzed"}

    def decide(self, observations):
        # Make decisions based on observations
        return {"action": "refactor", "target": "legacy_code.py"}

    def act(self, decision):
        # Execute decision
        # Agent modifies code here
        pass

    def reflect(self, outcome):
        # Learn from outcomes
        # Update internal state
        pass
```

Deploying the Agent:

```
# Spawn initial variant
waft spawn --agent RefactorAgent --mutation initial_config.json

# Evaluate fitness
waft eval --agent RefactorAgent

# Check results
waft stats
waft chronicle
```

DAILY WORKFLOWS

Morning Protocol: Project Bootstrap

Standard Morning Routine:

```
# 1. Navigate to project
cd my_laboratory

# 2. Check project status
waft info

# 3. Load epistemic context
waft session bootstrap

# 4. Review overnight changes
waft chronicle --limit 20

# 5. Check epistemic state
waft assess
```

1. **Step 1:** Verify substrate integrity with `waft verify`
2. **Step 2:** Load session context to restore epistemic state
3. **Step 3:** Review Flight Recorder logs for overnight agent activity
4. **Step 4:** Check moon phase indicator for epistemic health

Development Workflow: Agent Evolution Cycle

Complete Evolutionary Cycle:

```
# Phase 1: Spawn Variants
waft spawn --agent RefactorAgent --mutation improved_prompt.json
waft spawn --agent RefactorAgent --mutation better_error_handling.json
waft spawn --agent RefactorAgent --mutation optimized_algorithm.json

# Phase 2: Fitness Evaluation
waft eval --agent RefactorAgent --batch-size 3

# Phase 3: Evolutionary Selection
waft evolve --agent RefactorAgent --generation 5

# Phase 4: Analysis
waft stats
waft chronicle
waft assess
```

⚠ TELEPORT MASSIVE SAFETY PROTOCOL

Always run safety gates before major evolutionary steps. Use `waft check` to verify operations are safe. Agents with fitness < 0.5 are automatically marked as DEATH and will not continue evolving.

Documentation Workflow: Self-Documentation

WAFT Documenting Itself:

```
from waft import PDF

# Generate project documentation
PDF.from_template(
    template="field_guide",
    title="My Project Documentation",
    content=project_docs_html
).save("docs/project_guide.pdf")

# Generate scientific paper
PDF.scientific_paper(
    title="Agent Evolution Study",
    abstract="We studied agent evolution...",
    content=research_content
).save("research/evolution_study.pdf")
```

SITE-DELTA-9 INNOVATION

WAFT can observe its own codebase and generate documentation about itself. This recursive self-documentation creates a feedback loop where documentation informs development, which creates new features, which are documented using WAFT itself.

Epistemic Tracking Workflow

Logging Knowledge:

```
# Log discoveries
waft finding log "OAuth2 uses token refresh pattern" --impact 0.8
waft finding log "Database connection pooling improves performance" --impact 0.9

# Log knowledge gaps
waft unknown log "Need to investigate token expiration handling"
waft unknown log "Unclear how to handle rate limiting"

# Check epistemic state
```

```
waft assess

# View dashboard
waft dashboard
```

Safety Gates:

```
# Before major changes
waft check

# Returns: PROCEED, HALT, BRANCH, or REVISE
# - PROCEED: Safe to continue autonomously
# - HALT: Requires human approval
# - BRANCH: Spawn investigation before proceeding
# - REVISE: Modify approach and resubmit
```

ADVANCED TECHNIQUES

Multi-Agent Coordination

Coordinating Multiple Agents:

```
# Spawn specialized agents
waft spawn --agent RefactorAgent --mutation refactor_focus.json
waft spawn --agent TestAgent --mutation test_focus.json
waft spawn --agent DocAgent --mutation doc_focus.json

# Evaluate all agents
waft eval --agent RefactorAgent
waft eval --agent TestAgent
waft eval --agent DocAgent

# Select best variant for each
waft evolve --agent RefactorAgent
waft evolve --agent TestAgent
waft evolve --agent DocAgent
```

TELEPORT MASSIVE MULTI-AGENT PROTOCOL

When coordinating multiple agents, ensure they don't conflict. Use the Flight Recorder to track interactions. Monitor fitness scores to identify which agent combinations work best together.

Custom Fitness Functions

Creating Custom Scint Types:

```
# Define custom Scint type
from waft.gym.scint import ScintType, ScintQuest

class CUSTOM_SCINT(ScintType):
    # Custom reality fracture type
    name = "custom_scint"
    description = "Tests custom functionality"

# Create custom quest
quest = ScintQuest(
    scint_type=CUSTOM_SCINT,
    scenario="Test scenario with intentional errors",
    expected_stabilization="Correct error handling"
)

# Evaluate agent
fitness = agent.evaluate_fitness([quest])
```

Phylogenetic Analysis

Analyzing Agent Lineage:

```
# Load Flight Recorder data
from waft.core.science.observer import TheObserver

observer = TheObserver()
lineage = observer.get_lineage(genome_id="a4c426d8...")

# Analyze evolutionary path
for event in lineage:
    print(f"Generation {event.generation}: {event.event_type}")
    print(f"Fitness: {event.fitness.overall}")
    print(f"Mutations: {event.payload.mutations}")
```

> **SITE-DELTA-9 RESEARCH CAPABILITY**
>
> The Flight Recorder enables complete phylogenetic tree reconstruction. This allows scientific analysis of which mutations improve fitness, how agents converge on optimal solutions, and why certain lineages become evolutionary dead ends.

Hot-Swapping Genomes

Adopting Better Variants Mid-Execution:

```
# Agent discovers better variant
if variant_fitness > current_fitness:
    agent.hot_swap_genome(variant_genome_id)
    print(f"Evolved to genome {variant_genome_id}")
    print(f"Fitness improved: {current_fitness} → {variant_fitness}")
```

BEST PRACTICES

TELEPORT MASSIVE Operational Standards

SITE-DELTA-9 BEST PRACTICES

- ☐ ✅ Always create epistemic sessions before work
- ☐ ✅ Run safety gates before major evolutionary steps
- ☐ ✅ Log all discoveries and knowledge gaps
- ☐ ✅ Monitor fitness scores regularly
- ☐ ✅ Review Flight Recorder logs weekly
- ☐ ✅ Use version control for all code changes
- ☐ ✅ Document agent behavior and mutations
- ☐ ✅ Test agents in Scint Gym before deployment

Agent Design Principles

1. Single Responsibility
- Each agent should have one clear purpose
- Avoid agents that do too many things
- Specialized agents evolve faster

2. Observable Behavior
- All actions should be logged
- Fitness metrics should be measurable
- Mutations should be traceable

3. Evolutionary Fitness
- Design agents that can improve through mutation
- Avoid hard-coded solutions
- Enable genetic variation

4. Safety First
- Always run safety gates
- Test in Scint Gym before production
- Monitor for harmful mutations

> ⚠ **TELEPORT MASSIVE SAFETY PROTOCOL**
>
> Agents with fitness < 0.5 are automatically marked as DEATH. This prevents evolutionary dead ends from consuming resources. Always monitor fitness scores and intervene if agents are trending toward DEATH.

Memory Management

Organizing _pyrite Structure:

```
_pyrite/
├── active/          # Current work (keep < 10 items)
│   ├── task_001.md
│   └── task_002.md
├── backlog/         # Future work (prioritize regularly)
│   ├── idea_001.md
│   └── idea_002.md
├── standards/       # Project standards (reference only)
│   ├── code_style.md
│   └── architecture.md
└── gym_logs/        # Fitness results (archive monthly)
    └── evaluation_*.jsonl
```

Best Practices:
- Keep `active/` focused (max 10 items)
- Review `backlog/` weekly
- Archive `gym_logs/` monthly
- Update `standards/` as project evolves

TROUBLESHOOTING

Common Issues and Solutions

Issue: Agent Fitness Declining

```
# Check recent mutations
waft chronicle --agent RefactorAgent --limit 20

# Review Flight Recorder logs
```

```
cat _pyrite/science/laboratory.jsonl | grep RefactorAgent | tail -20

# Spawn new variants with different mutations
waft spawn --agent RefactorAgent --mutation conservative_approach.json
```

TELEPORT MASSIVE DIAGNOSTIC PROTOCOL

If agent fitness consistently declines, the mutation strategy may be too aggressive. Try spawning variants with more conservative mutations. Review the phylogenetic tree to identify which mutations caused the decline.

Issue: Epistemic State Unclear

```
# Check epistemic vectors
waft assess --history

# Review findings and unknowns
waft session status

# Log missing knowledge explicitly
waft unknown log "Specific knowledge gap description"
```

Issue: Project Verification Fails

```
# Check project structure
waft verify --verbose

# Reinitialize if needed
waft init

# Verify dependencies
waft sync
```

Issue: PDF Generation Errors

```
# Check WeasyPrint installation
python3 -c "import weasyprint; print('OK')"

# Try different template
PDF.from_template(template="lab_notes", ...)

# Use evolution system instead
PDF.from_content(content=markdown_content, style="clinical_standard")
```

REAL-WORLD EXAMPLES

Example 1: Code Refactoring Agent

Scenario: Automatically refactor legacy code while maintaining functionality.

```
# 1. Create agent
waft new refactor_laboratory
cd refactor_laboratory

# 2. Define RefactorAgent
# (see First Agent Deployment section)

# 3. Spawn variants
waft spawn --agent RefactorAgent --mutation improved_ast_parsing.json
waft spawn --agent RefactorAgent --mutation better_naming_conventions.json

# 4. Evaluate fitness
waft eval --agent RefactorAgent

# 5. Evolve to best variant
waft evolve --agent RefactorAgent

# 6. Deploy evolved agent
# Agent now has improved refactoring capabilities
```

SITE-DELTA-9 SUCCESS STORY

RefactorAgent evolved from 0.65 fitness to 0.89 fitness over 12 generations. Key mutations included improved AST parsing, better naming convention detection, and enhanced code structure analysis. The agent now handles 40% more code patterns than the original variant.

Example 2: Documentation Generator

Scenario: Automatically generate comprehensive project documentation.

```
from waft import PDF
from waft.reflection import ReflectionSystem

# 1. Observe codebase
reflection = ReflectionSystem()
gaps = reflection.analyze_documentation_gaps()

# 2. Generate documentation
for gap in gaps:
    content = generate_doc_content(gap)
```

```
    PDF.from_template(
        template="field_guide",
        title=gap.title,
        content=content
    ).save(f"docs/{gap.filename}")

# 3. Document the documentation system
PDF.from_template(
    template="lab_notes",
    title="Documentation Generation Process",
    content=reflection.to_markdown()
).save("docs/doc_generation_process.pdf")
```

Example 3: Test Generation Agent

Scenario: Automatically generate test cases for code.

```
# 1. Create TestAgent
waft spawn --agent TestAgent --mutation unit_test_focus.json

# 2. Evaluate on codebase
waft eval --agent TestAgent --quest-type LOGIC_FRACTURE

# 3. Evolve based on test coverage metrics
waft evolve --agent TestAgent --generation 10

# 4. Deploy evolved TestAgent
# Agent now generates comprehensive test suites
```

TELEPORT MASSIVE Recommendation

Start with simple agents and let them evolve. Don't try to build the perfect agent from scratch. Instead, create a basic agent, spawn variants, evaluate fitness, and let evolution find the optimal solution. This is the WAFT way: making the impossible, inevitable.

CONCLUSION: MAKING THE IMPOSSIBLE, INEVITABLE™

WAFT enables you to breed AI agents that evolve, adapt, and improve. Through the Three Pillars—the Substrate, the Physics, and the Flight Recorder—WAFT provides a complete system for directed evolution of self-modifying AI agents.

Remember:
- **Don't just build agents. Breed them.**
- **Let evolution find the optimal solution.**
- **Track everything for scientific analysis.**
- **Make the impossible, inevitable.**

TELEPORT MASSIVE MISSION STATEMENT

WAFT is not just a framework—it's a scientific instrument for studying the physics of artificial cognition. Every agent evolution, every mutation, every fitness evaluation contributes to our understanding of how AI can improve itself. The ultimate goal: observe a "God-Head" agent emerge from thousands of generations of directed mutation and selection.

Generated: January 18, 2026 at 08:53 PM
WAFT Version: 0.3.1-alpha
Handbook Version: 2.0
Document ID: TM-OPMAN-WAFT-001
Classification: INTERNAL USE ONLY
Facility: Site-Delta-9 (WAFT Development Laboratory)
Tagline: Making the Impossible, Inevitable™